FraudFlow Technical Writeup

FraudFlow — Technical Writeup

What it is: A real-time payments fraud detection pipeline built from scratch as a data engineering portfolio project. Simulated card transactions flow through Kafka into a PySpark Structured Streaming medallion architecture (bronze → silver → gold Delta tables), with fraud signals computed per card, alerting via AWS Lambda + EventBridge + SES, and monitoring via Prometheus + Grafana. The full pipeline also runs on Databricks Free Edition.

---

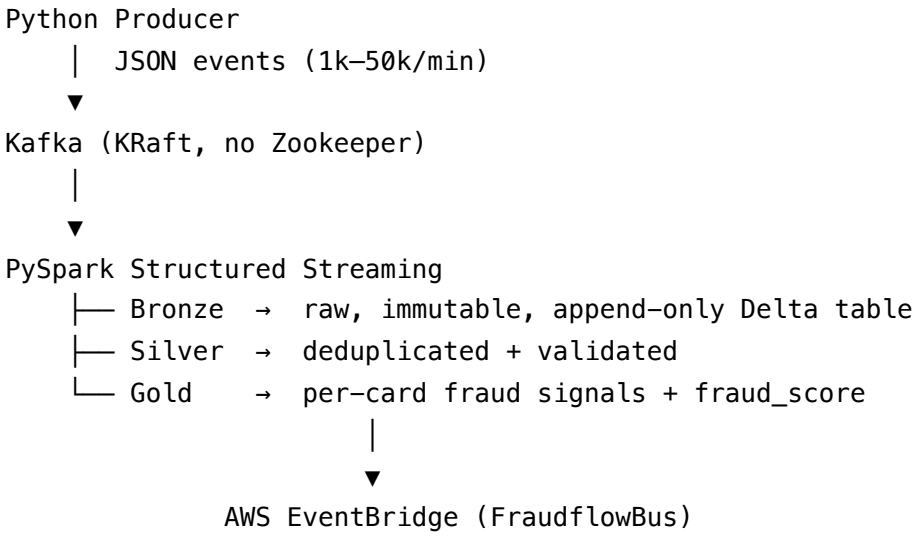
1. Problem Statement

Card fraud detection is one of the canonical problems in streaming data engineering. The challenge is not detection accuracy (that's an ML problem) but *pipeline design*: how do you process millions of events per day, deduplicate them reliably, compute per-card aggregations in near real-time, and fire alerts within seconds — all without losing data or duplicating alerts?

This project builds the data engineering layer of that system. The fraud scoring is intentionally rule-based (z-scores, velocity counts, geo distance) rather than ML-based so the architecture remains the focus.

---

2. Architecture Overview



```
                    │
                    ▼
              Lambda → SES email alert

Prometheus ← producer /metrics
Grafana    ← Prometheus
```

The same silver and gold logic runs on Databricks Free Edition — replacing Kafka with a self-contained data generator notebook, and swapping `processingTime` triggers for `availableNow`.

---

3. Technology Decisions

3.1 Kafka over alternatives

Kafka was chosen over RabbitMQ, Redis Streams, or a simple database queue for three reasons:

1. **Industry standard for streaming pipelines.** Real payments systems (Stripe, Square, Adyen) use Kafka or Kafka-compatible systems. Portfolio credibility matters.
2. **Consumer group semantics.** Multiple Spark jobs (bronze, silver, gold) can each maintain their own committed offset independently. With a queue like RabbitMQ, a message is consumed once — you can't have three independent consumers at different positions.
3. **Spark Structured Streaming has native Kafka integration.** `spark.readStream.format("kafka")` is a first-class connector with checkpointing built in.

3.2 KRaft mode (no Zookeeper)

Kafka has supported running without Zookeeper since 3.3 (production-ready). The decision to use KRaft:

- **One fewer service.** Zookeeper adds a separate container, separate port, and separate failure mode.
- **Modern practice.** Zookeeper mode is deprecated as of Kafka 3.7 and will be removed in a future release. New projects should not use it.
- **Simpler mental model.** KRaft means the broker is also the controller — no split-brain between broker state and Zookeeper state.

The tradeoff is that the env var setup is more verbose (you must set `KAFKA_NODE_ID`, `CLUSTER_ID`, `KAFKA_PROCESS_ROLES`, and configure three listeners manually). This is worth it.

3.3 `confluent-kafka` over `kafka-python`

Two Python Kafka client libraries exist:

	confluent-kafka	kafka-python
Under the hood	librdkafka (C)	Pure Python
GIL impact	Network I/O in C threads, GIL-free	Every send holds the GIL
Throughput at 50k events/min	Fine	Becomes a bottleneck
Binary wheel	Yes (requires glibc, not musl)	Pure Python, works anywhere

At 1k events/min the difference is negligible. At 50k events/min, the pure Python client becomes the bottleneck. Since this project supports up to 50k events/min, `confluent-kafka` was the right choice. The consequence: the Docker base image must be `python:3.12-slim` (Debian/glibc), not Alpine (musl libc).

3.4 JSON over Avro

Avro with a Schema Registry is the production standard for Kafka message serialization — it enforces schema evolution, is compact, and self-describes. JSON was chosen here because:

1. **No schema registry to run.** Avro requires Confluent Schema Registry as an additional service, adding complexity before the interesting parts.
2. **Human readable.** You can `kcat` a topic and read the events directly.
3. **Spark reads it with `from_json()`** and an explicit StructType — no additional libraries.

The comment in `producer.py` notes that production would use Avro + Schema Registry. The schema is explicitly defined as a StructType in `bronze_ingestion.py` rather than inferred — schema inference on Kafka JSON is unreliable and slow.

3.5 Delta Lake over Parquet

Plain Parquet files would work for batch analytics, but Delta Lake adds:

- **ACID transactions.** Two Spark jobs writing simultaneously won't corrupt the table.
- **Schema enforcement.** Writes that don't match the table schema are rejected.
- **`DESCRIBE HISTORY`.** Every write is logged — useful for debugging and auditing (demonstrated in `04_explore.py`).
- **Streaming source support.** `spark.readStream.format("delta")` lets silver read from bronze as a stream without polling hacks.
- **`dropDuplicates` with watermarks.** The Delta streaming source tracks which versions of the table have been read, enabling exactly-once-style deduplication when combined with

Spark's watermark mechanism.

3.6 Prometheus + Grafana over other monitoring

The producer exposes a `/metrics` endpoint using `prometheus_client`. Prometheus scrapes it every 15 seconds; Grafana visualizes it. This was chosen because:

1. The pull model (Prometheus scrapes the producer) works perfectly in Docker Compose without any additional configuration inside the producer beyond exposing the endpoint.
2. Grafana dashboard JSON can be pre-provisioned as a file — no manual UI setup required on first boot.
3. Prometheus + Grafana is the dominant open-source observability stack, appearing in nearly every data engineering interview discussion about production monitoring.

---

4. Producer Design Decisions

4.1 Dynamic fraud rate controller

Naively randomly injecting fraud at 1.5% would cause the realized rate to drift due to the velocity burst pattern (which fires many events in a row regardless of the rate). A simple proportional controller was implemented in `FraudInjector.decide()`:

```
current_rate = fraud_count / total_count
adjustment = target * (1 + (target - current_rate) / target)
injection_probability = clamp(adjustment, 0, 1)
```

If the realized rate is below target, `injection_probability` exceeds `target` to compensate. If above, it's suppressed. Velocity bursts bypass this check intentionally — a burst is a single fraud event that happens to generate many transactions; stopping it mid-burst would be unrealistic.

4.2 Velocity window pruning

`CardProfile.velocity_window` stores timestamps of recent transactions for a card. This is pruned to the last 60 seconds at the *start* of each call to `generate_transaction()`, not at the end. If pruned at the end, a card that fires a velocity burst would accumulate thousands of entries in the window before any are pruned. Pruning at the start keeps the list bounded even under burst conditions.

4.3 `producer.poll(0)` in the main loop

The `confluent-kafka` producer is asynchronous — `produce()` enqueues a message, but delivery callbacks (`on_delivery`) only fire when you call `poll()`. This is a common source of bugs: the producer seems to work (no exceptions) but delivery errors are silently swallowed. `poll(0)` is called on every loop iteration to drain the callback queue without blocking.

4.4 Three listeners in Kafka

```
PLAINTEXT  :9092  → inter-broker + Spark (inside Docker network)
CONTROLLER :9093  → KRaft internal only, never advertised
EXTERNAL   :9094  → host clients (kcat, console consumer, local testing)
```

The `CONTROLLER` listener is deliberately excluded from `ADVERTISED_LISTENERS`. If it's included, Kafka crashes on startup because clients would attempt to connect to the controller port for metadata requests, which it doesn't handle.

---

5. Streaming Design Decisions

5.1 Bronze: raw and immutable

Bronze ingests from Kafka with no transformation — only schema parsing and adding `bronze_ingest_time`. The reasoning: if silver or gold logic turns out to be wrong, you can always re-derive them from bronze. If bronze itself transforms data and the transformation was wrong, you've lost information. Raw data is the source of truth.

`startingOffsets="earliest"` ensures no events are skipped on restart. `maxOffsetsPerTrigger=10_000` caps how many Kafka messages are read per micro-batch, preventing a single batch from reading the entire topic backlog on startup.

5.2 Silver: why watermarking is necessary for deduplication

`dropDuplicates(["transaction_id", "event_time"])` sounds simple, but in a streaming context Spark has to *remember* every transaction_id it has ever seen to reject future duplicates. Without a bound, that state grows forever — a job running for days would eventually OOM.

`.withWatermark("event_time", "10 minutes")` tells Spark: "events more than 10 minutes behind the latest event seen can be dropped from deduplication state." This trades a small amount of correctness (very late duplicates will slip through) for bounded memory. In a payments system, a duplicate arriving more than 10 minutes late would be rejected by the downstream idempotency layer anyway.

The watermark must be applied *before* `dropDuplicates`, and the watermark column must be included in the `dropDuplicates` key — Spark requires this to correctly bound the state.

5.3 Gold: `foreachBatch` instead of native streaming aggregations

The gold layer needs to: (1) compute per-card statistics, (2) join those statistics back to individual transactions, and (3) optionally call the AWS EventBridge API from the driver. None of this fits the native `groupBy().agg()` streaming model cleanly.

`foreachBatch` gives you the full batch DataFrame API inside each micro-batch. The tradeoff: you lose Spark's built-in state management for windowed aggregations across batches. That's acceptable here because fraud scoring is done within each micro-batch (we care about velocity within the *current* batch window, not across all time), and the external API call (EventBridge) has to run on the driver anyway — Spark workers don't have AWS credentials.

EventBridge `put_events` accepts up to 10 entries per call, so alerts are batched in groups of 10 inside `foreachBatch`.

5.4 Checkpointing

Every streaming query has a `checkpointLocation`. The checkpoint stores two things: the committed Kafka offsets (so the job doesn't re-read messages it already processed) and the streaming query's internal state (watermark position, deduplication state). Without checkpointing, a job restart re-reads from `startingOffsets="earliest"` and reprocesses everything.

Checkpoints are stored in the same Delta base path as the tables, so they're in the same storage system and survive container restarts.

5.5 `wait_for_delta_table()` — startup race condition

Silver tries to `readStream` from the bronze Delta table. If silver starts before bronze has written its first micro-batch, the Delta table doesn't exist yet and Spark throws `DELTA_SCHEMA_NOT_SET`. The fix is a polling loop using `DeltaTable.isDeltaTable(path)` — it retries every 10 seconds for up to 180 seconds. The same fix was applied to gold waiting for silver.

---

6. AWS Alerting Design Decisions

6.1 Custom EventBridge bus

A custom bus (`FraudflowBus`) was created rather than using the default bus for two reasons:

1. **Isolation.** Events from other AWS services (CodePipeline, EC2, etc.) flow through the default bus. Using the default bus means your fraud rules might accidentally match unrelated events.
2. **SAM shorthand limitation.** SAM's `Events:` shorthand in a `AWS::Serverless::Function` only works with the default bus. Custom bus wiring requires explicit `AWS::Events::Rule` + `AWS::Lambda::Permission` resources, which is more code but also more transparent about what's happening.

6.2 `AWS_REGION_OVERRIDE` not `AWS_REGION`

`AWS_REGION` is a reserved environment variable in the Lambda runtime — CloudFormation will reject the template if you try to set it. The Lambda handler reads `AWS_REGION_OVERRIDE` and falls back to `boto3`'s default region resolution (which picks up the Lambda execution region automatically anyway).

6.3 `FraudAlertFunctionPermission`

Without an explicit `AWS::Lambda::Permission` granting `events.amazonaws.com` the right to invoke the Lambda, EventBridge silently accepts the `put-events` call, matches the rule, and then drops the event without invoking Lambda. No error is returned. This was the most counterintuitive pitfall in the AWS setup — always add the permission resource, always scope it to the specific rule ARN via `SourceArn`.

6.4 `Detail` must be a JSON-encoded string

The EventBridge `put-events` API accepts `Detail` as a string. If you pass a Python dict directly (which `boto3` serializes to JSON), it works. But if you pass a nested object by accident (e.g. a dict that contains another dict already serialized as a string), rule matching silently fails. The convention enforced throughout is `"Detail": json.dumps(detail_dict)` — always an explicit `json.dumps`, never assumed.

---

7. Databricks Portability Decisions

7.1 `DELTA_BASE_PATH` as the single portability lever

All path construction flows through `BASE_PATH` in `00_setup.py`. This means moving the pipeline from local Docker to Databricks to S3 to Azure ADLS requires changing exactly one value. The

design was intentional from the start — every path is a derivation of `BASE_PATH`, never hardcoded.

7.2 `trigger(availableNow=True)` on Databricks

Local Docker jobs run with `trigger(processingTime="10 seconds")` and loop forever. On Databricks (especially Free Edition), notebook cells must terminate — a cell that runs forever blocks the notebook and eventually times out. `availableNow=True` processes all data currently in the source table and then stops cleanly. The cell finishes, the results are visible, and you can re-run to pick up new data.

7.3 Unity Catalog auto-detection

Databricks Free Edition enforces Unity Catalog — raw `dbfs:/` paths are blocked. The fix in `00_setup.py` auto-detects the current catalog using `SELECT current_catalog()`, then creates a `fraudflow` schema and `data` volume if they don't exist, and constructs `BASE_PATH = /Volumes/<catalog>/fraudflow/data`. The DBFS fallback is retained for Community Edition users.

7.4 Self-contained data generator on Databricks

Databricks Free Edition clusters sit behind NAT — they cannot reach a Kafka broker running on your laptop. Rather than requiring a cloud Kafka cluster (Confluent Cloud, MSK), `01_data_generator.py` contains a self-contained copy of the core producer logic (minus the Kafka send) and writes directly to the bronze Delta table. The fraud patterns and card/merchant simulation are identical to the local producer. This keeps the Databricks setup dependency-free.

---

8. Key Challenges and Solutions

8.1 `bitnami/kafka:3.7` tag did not exist

Bitnami does not publish floating minor-version tags (`3.7`) — only full version tags (`3.7.0`). Worse, the `bitnami/kafka` image uses `KAFKA_CFG_*` env var naming, but the official `apache/kafka` image uses unprefixed `KAFKA_*` naming. Switching to `apache/kafka:3.7.0` required rewriting all env vars, generating a `CLUSTER_ID` with `kafka-storage.sh random-uuid`, and updating the volume path and health check command.

8.2 Silver `DELTA_SCHEMA_NOT_SET` on startup

Silver's `readStream` from the bronze Delta table failed when silver started before bronze had written its first micro-batch (the table didn't exist yet). Fixed with a `wait_for_delta_table()`

polling function. The root cause is that in Docker Compose, all three Spark containers start roughly simultaneously — `restart: on-failure` was also added so the container retries if it loses the race.

8.3 Gold `JAVA_GATEWAY_EXITED` (out of memory)

Three simultaneous PySpark JVMs each configured with `spark.driver.memory=2g` exceeded available RAM (6GB total, leaving nothing for the OS and Docker overhead). Reduced to `1g` per job. The fix was in `spark_utils.py` (one place) because all three jobs use the same `build_spark_session()` factory.

8.4 Grafana "No data" on all panels

The pre-provisioned Grafana dashboard JSON referenced datasource UID `"prometheus"`, but Grafana auto-generated a random UID (`PBFA97CFB590B2093`) for the provisioned datasource. Fixed by adding `uid: prometheus` to the datasource provisioning YAML. Verified by calling the Grafana API (`GET /api/datasources`) to inspect the actual UID before and after the fix.

8.5 Databricks Free Edition DBFS restriction

`dbfs:/fraudflow` paths are blocked on Free Edition (Unity Catalog enforcement). Fixed by auto-detecting the Unity Catalog catalog in `00_setup.py` and constructing a `/Volumes/` path. The `DESCRIBE HISTORY %sql` cell in `04_explore.py` also had a hardcoded DBFS path — replaced with a `%python` cell using an f-string so it picks up the dynamic `GOLD_PATH`.

---

9. Results

Pipeline validated end-to-end on Databricks Free Edition (50,000 synthetic transactions):

Layer	Rows	Notes
Bronze	50,000	Raw generated events
Silver	50,000	0 dropped (no duplicates in synthetic data)
Gold	50,000	Enriched with fraud signals
High confidence	261	fraud_score ≥ 0.70 (0.52%)

Fraud breakdown:

Pattern	Events	Avg Score	Avg Amount
`velocity_burst`	598	0.441	$46.79
`amount_spike`	252	0.790	$1,834.03
`impossible_travel`	129	0.645	$50.64

- Amount spikes average **39× the normal transaction value** ($1,834 vs $47)
- 248 transactions have z-score > 3 — all are amount spikes (the signal works)
- Fraud rate by country ranges from 1.53% (AU) to 2.59% (CA), close to the 1.5% target

Local Docker producer: tested at 1,000 events/min (default) through 50,000 events/min (configured max). Prometheus and Grafana dashboard confirmed live metrics at all throughput levels.

---

10. What I Would Do Differently at Production Scale

1. **Avro + Schema Registry.** JSON is convenient for development but schema drift is a real risk in production. Confluent Schema Registry with Avro (or Protobuf) would enforce compatibility.

2. **Separate checkpoints per environment.** Currently checkpoints live alongside data. In production, checkpoints belong in object storage (S3/GCS/ADLS) with versioned paths per deployment.

3. **ML scoring layer.** The rule-based fraud score (velocity + z-score + geo) is interpretable and fast, but a gradient-boosted model trained on labeled fraud data would significantly improve recall. The pipeline is designed to accommodate this — the gold layer is where the model inference step would go.

4. **Dead letter queue.** Events that fail schema validation in silver are silently dropped. A production system would route them to a DLQ (another Kafka topic or S3 prefix) for investigation.

5. **Idempotent Lambda.** The current Lambda sends one email per EventBridge event. At high fraud rates this could generate hundreds of emails per minute. Rate limiting (DynamoDB TTL-based dedup) or alert aggregation (SNS + digest Lambda) would be needed.

6. **Terraform over SAM.** SAM is convenient for Lambda-only stacks but Terraform would be more appropriate if the alerting infrastructure grew to include additional services.